

COM 3240

REINFORCEMENT LEARNING



You

create a painting with the title Artificial Neural Networks as function approximations in the style of Claude Monet, widescreen aspect ratio



DALL-E



Error creating image

I'm unable to generate images for the request you provided due to our content policy. If there's another way I can assist you or if you'd like to explore a different concept, please let me know!



You

create a painting with the title Artificial Neural Networks as function approximations in the style of Edvard Munch, widescreen aspect ratio



DALL-E



Error creating image

I'm unable to generate images for the request you provided due to our content policy. If you have another idea or would like to explore a different theme, please let me know!



FUTURE REWARDS

SARSA

$$q^{\pi}(s, a) = \mathbb{E}_{\pi}[r + \gamma q^{\pi}(s', a') \mid s, a]$$

$$\mathbb{E}_{\pi}[r + \gamma q^{\pi}(s', a') - q^{\pi}(s, a) \mid s, a] = 0$$

$$E = \frac{1}{2} \left(r + \gamma Q(s', a') - Q(s, a) \right)^2$$

FUTURE REWARDS

THE SARSA ALGORITHM

1. Initialise $Q(s, a)$ arbitrarily for all $s \in S$ and $a \in A(s)$.
2. Repeat (for each episode):
 - a. Initialise s .
 - b. Choose an action a from s using a policy derived from Q (e.g., ϵ -greedy).
 - c. Repeat (for each step of episode):
 - i. Take action a , observe reward r and next state s' .
 - ii. Choose a' from s' using policy derived from Q (e.g., ϵ -greedy).
 - iii. $Q(s, a) \leftarrow Q(s, a) + \eta * [r + \gamma * Q(s', a') - Q(s, a)]$.
 - iv. $s \leftarrow s'$; $a \leftarrow a'$.
 - d. until s is terminal.

On-policy

FUTURE REWARDS

Q-LEARNING

$$q^*(s, a) = \mathbb{E}[r + \gamma \max_{a'} q^*(s', a') \mid s, a]$$

$$\mathbb{E}[r + \gamma \max_{a'} q^*(s', a') - q^*(s, a) \mid s, a] = 0$$

$$E = \frac{1}{2} \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right)^2$$

FUTURE REWARDS

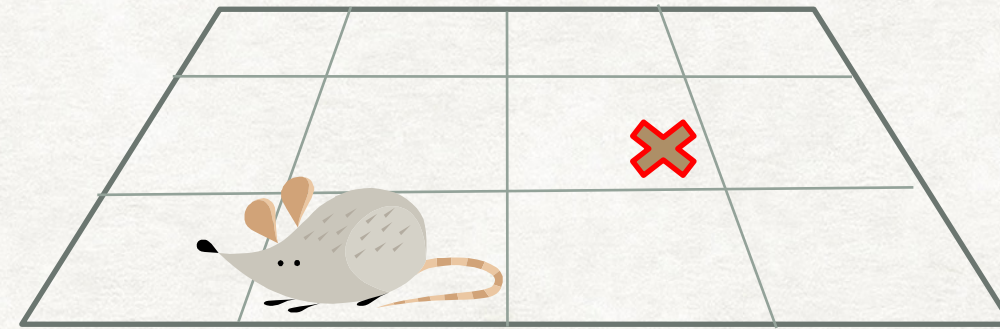
THE Q-LEARNING ALGORITHM

1. Initialise $Q(s, a)$ arbitrarily for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$.
2. Repeat (for each episode):
 - a. Initialise s .
 - b. Repeat (for each step of episode):
 - i. Choose an action a from s using a policy derived from Q (e.g., ϵ -greedy).
 - ii. Take action a , observe reward r and next state s' .
 - iii. $Q(s, a) \leftarrow Q(s, a) + \eta * [r + \gamma * \max_{a'} Q(s', a') - Q(s, a)]$.
 - iv. $s \leftarrow s'$.
 - c. until s is terminal.

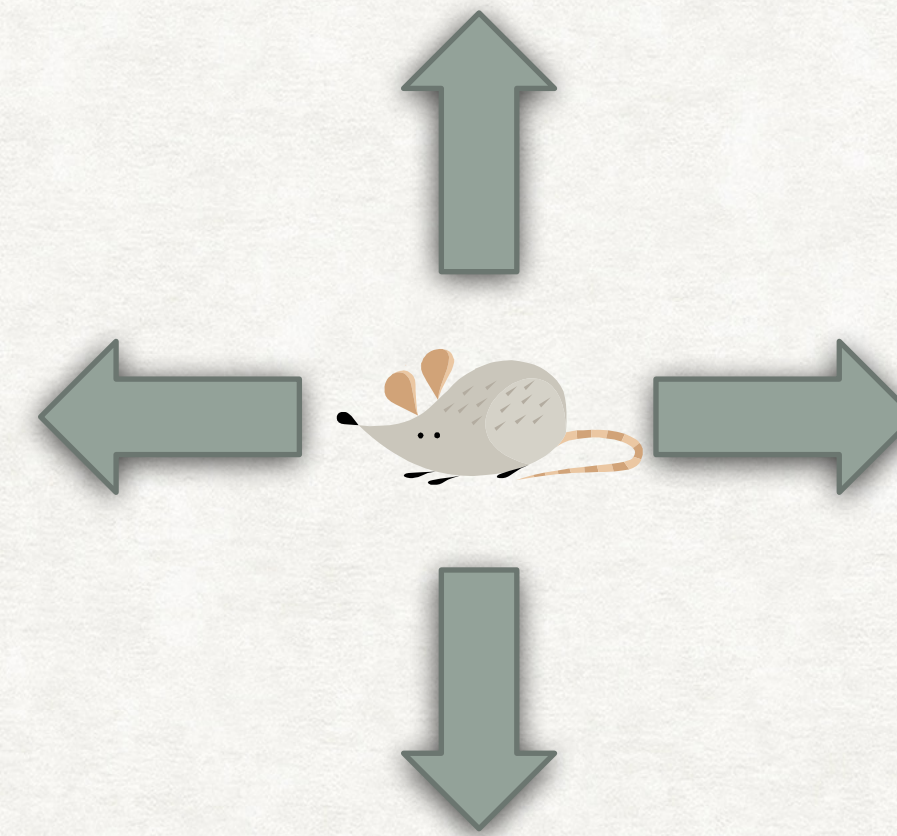
Off-policy

FUTURE REWARDS

Q-VALUES



Squares = State (in this case)



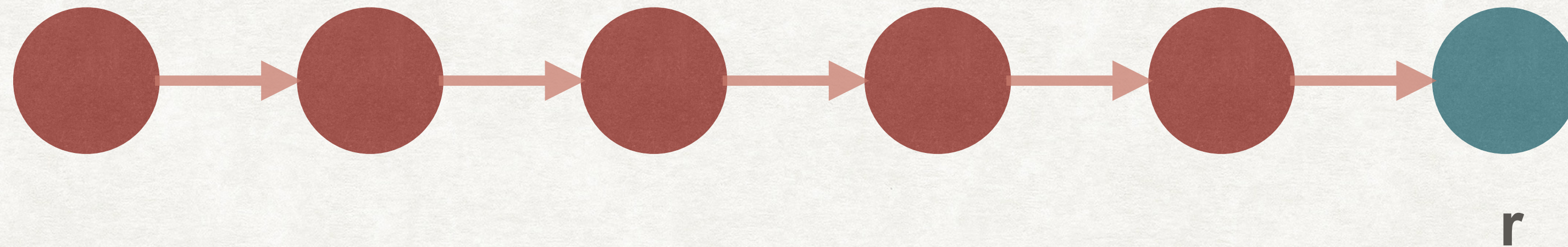
Actions

Q-values stored in a table

FUTURE REWARDS

SARSA

$$\Delta Q = \eta \left(r + \gamma Q(s', a') - Q(s, a) \right)$$



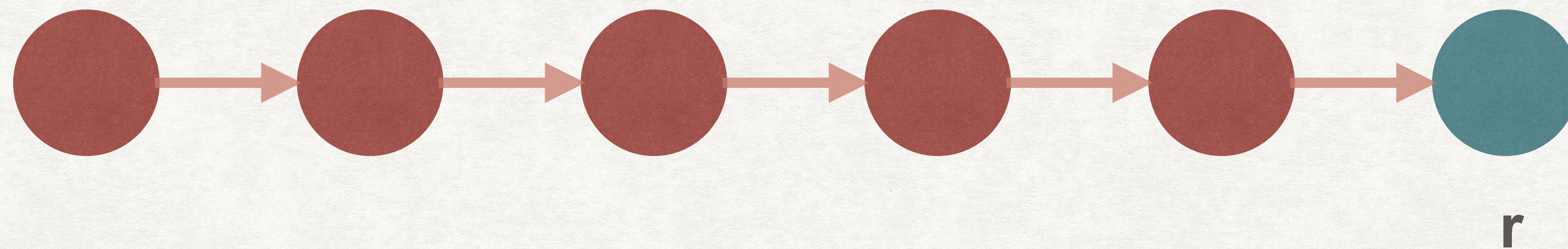
A robot walks this corridor and finds reward.

How many Q-values will be updated each time?

FUTURE REWARDS

SARSA

$$\Delta Q = \eta \left(r + \gamma Q(s', a') - Q(s, a) \right)$$

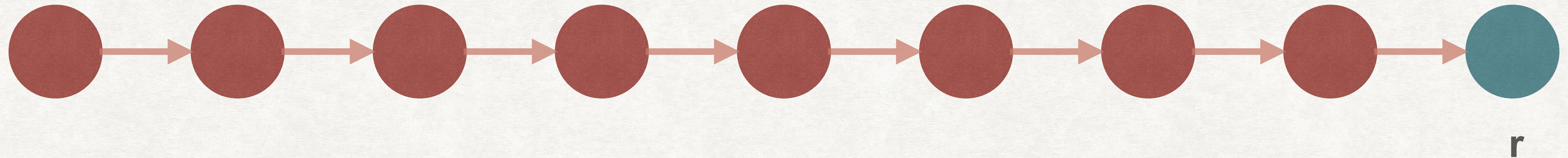


How large has to be the table storing the Q values?

FUTURE REWARDS

SARSA

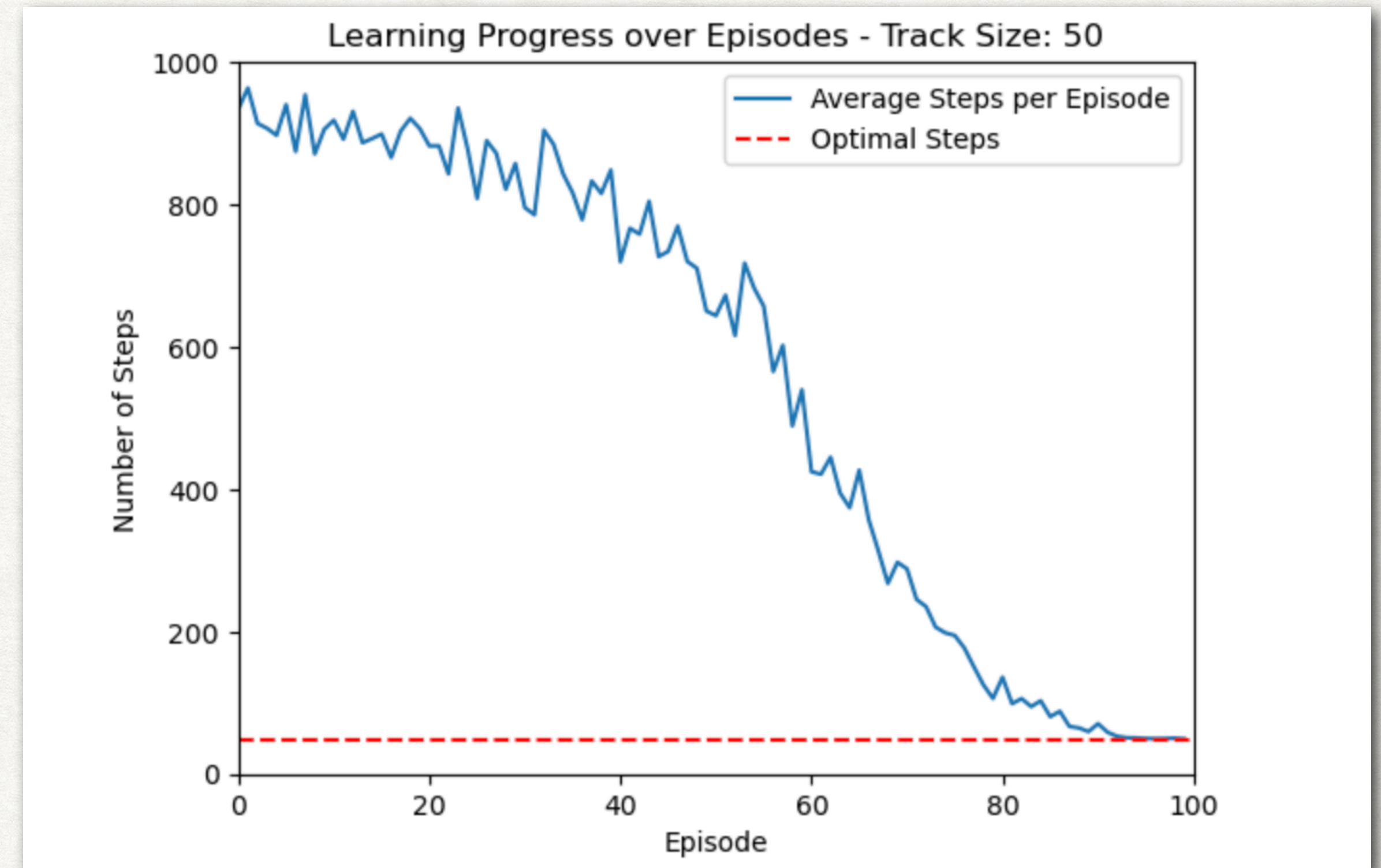
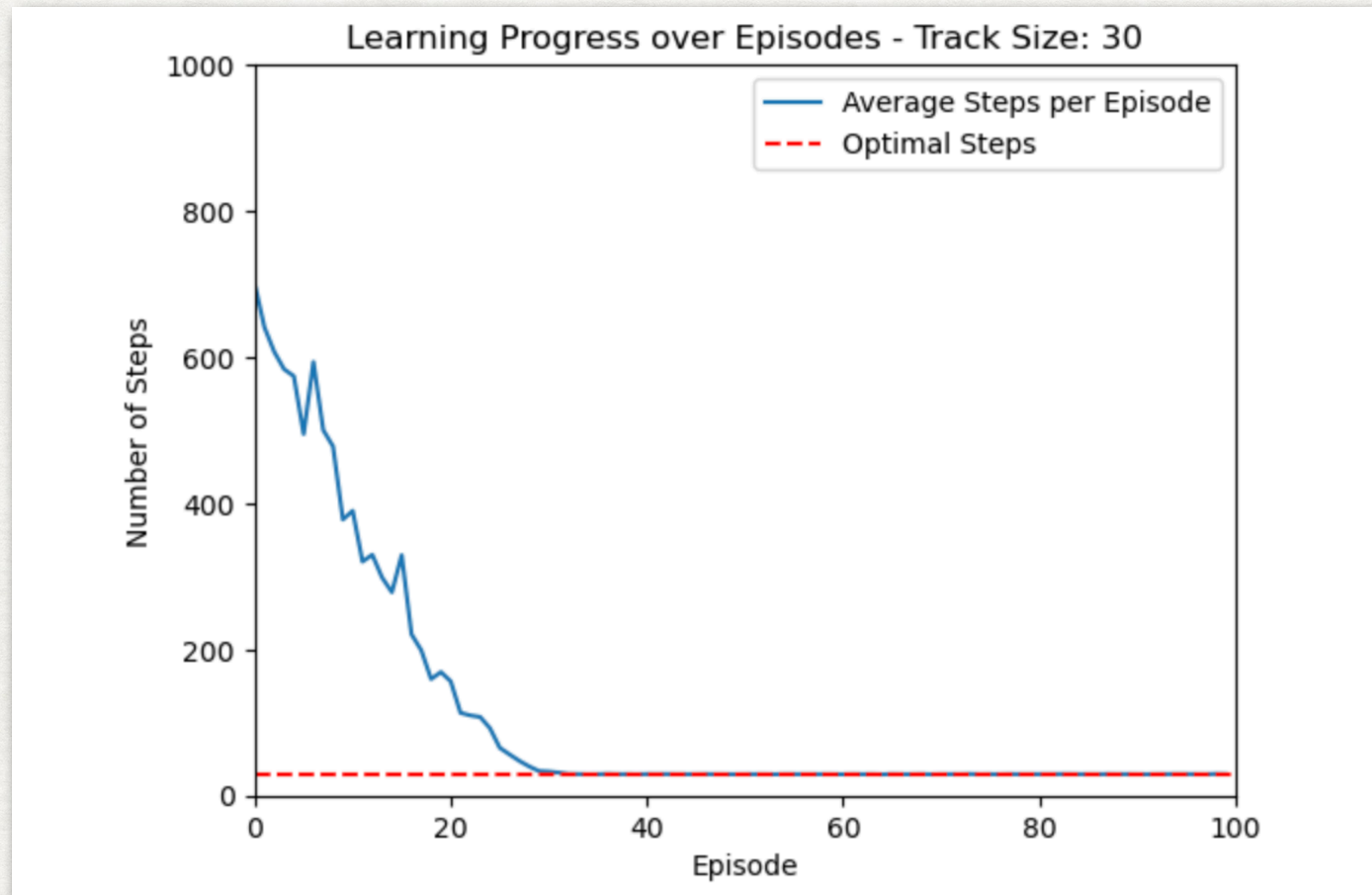
$$\Delta Q = \eta \left(r + \gamma Q(s', a') - Q(s, a) \right)$$



We increase the number of states. How many Q-values we need to store?

FUTURE REWARDS

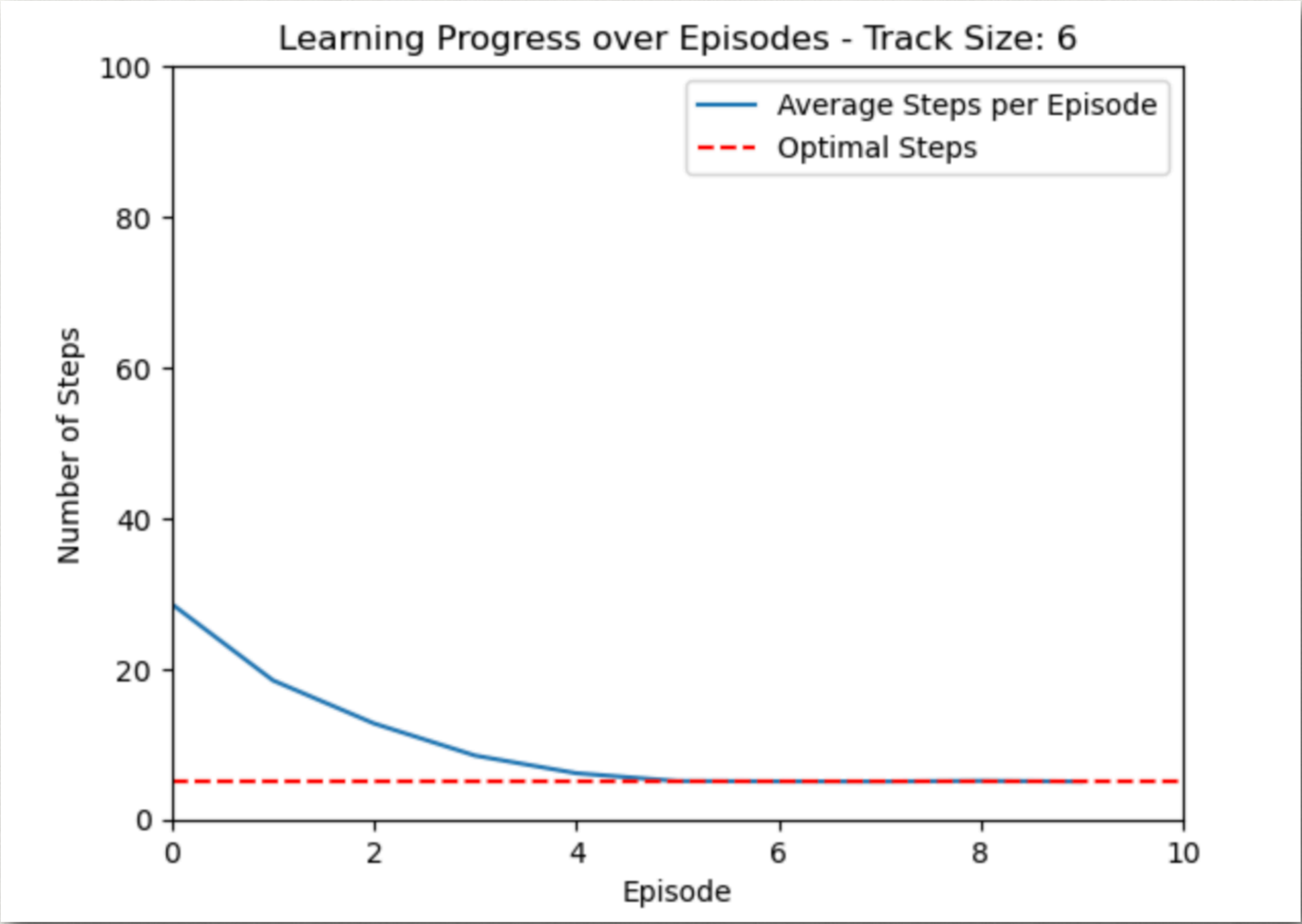
SARSA



Alternative metrics

FUTURE REWARDS

SARSA



Backward Forward

[0.000,	0.002,	]
[0.000,	0.012,	]
[0.000,	0.069,	]
[0.000,	0.261,	]
[0.000,	0.651,	]
[0.000,	0.000,	]

Has the algorithm converged?



create me a painting
with the title Chess, Go,
Backgammon and other
games



Ramon y Cahal

1mm³: 10,000 neurons 3km wires

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

SINGLE NEURON

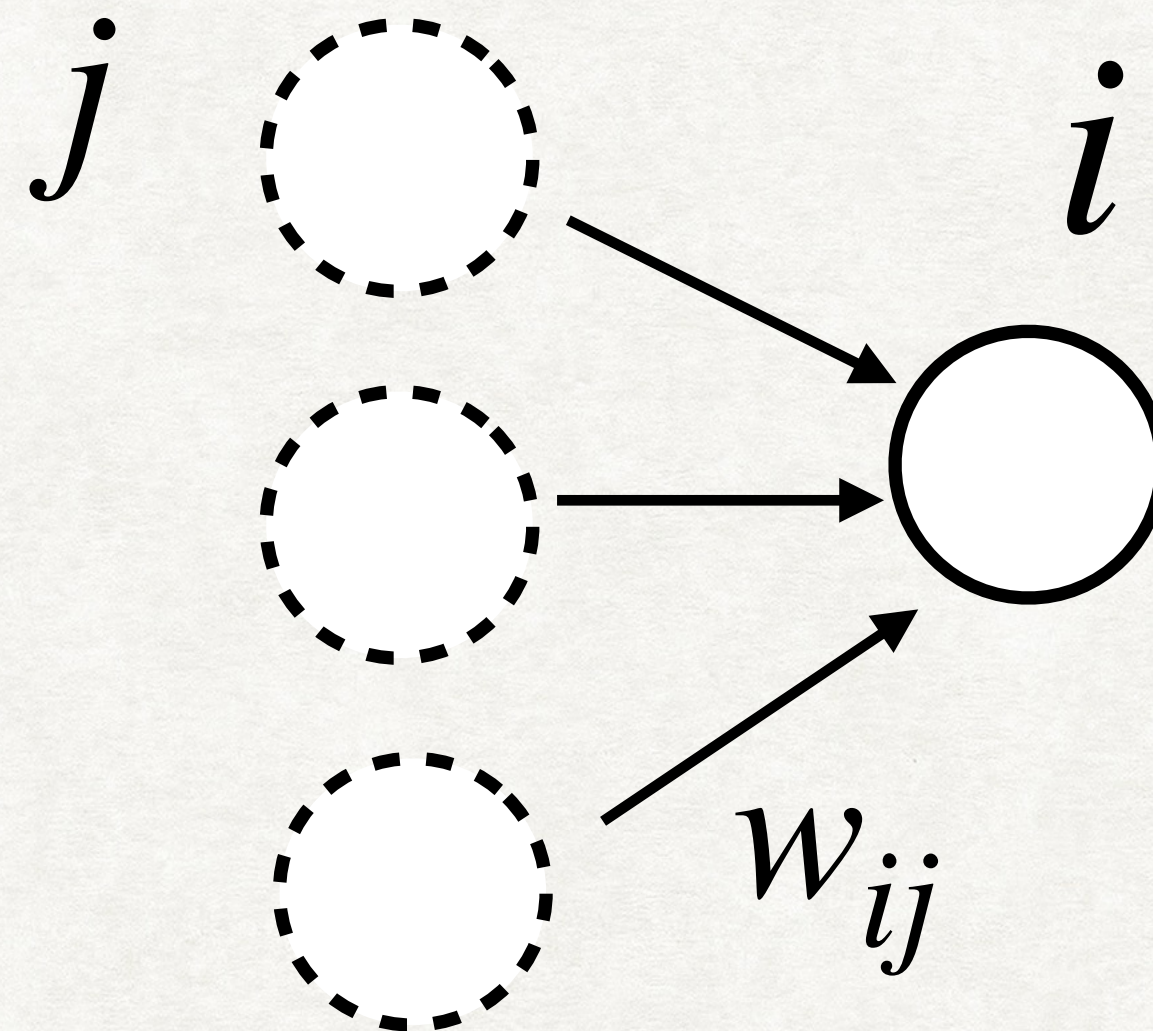
$$\mathbf{y} = F(\mathbf{x})$$

Samples from the input and output space of the function.

$$\mathbf{x}^{(1)} \quad \mathbf{x}^{(2)} \quad \mathbf{x}^{(3)} \dots \mathbf{x}^{(n)}$$

$$\mathbf{y}^{(1)} \quad \mathbf{y}^{(2)} \quad \mathbf{y}^{(3)} \dots \mathbf{y}^{(n)}$$

$$\hat{\mathbf{y}} = \hat{F}(\mathbf{x})$$



$$E = \frac{1}{2n} \sum_i \sum_{\mathbf{x}} \left(y_i^t(\mathbf{x}) - y_i(\mathbf{x}; \mathbf{W}) \right)^2$$

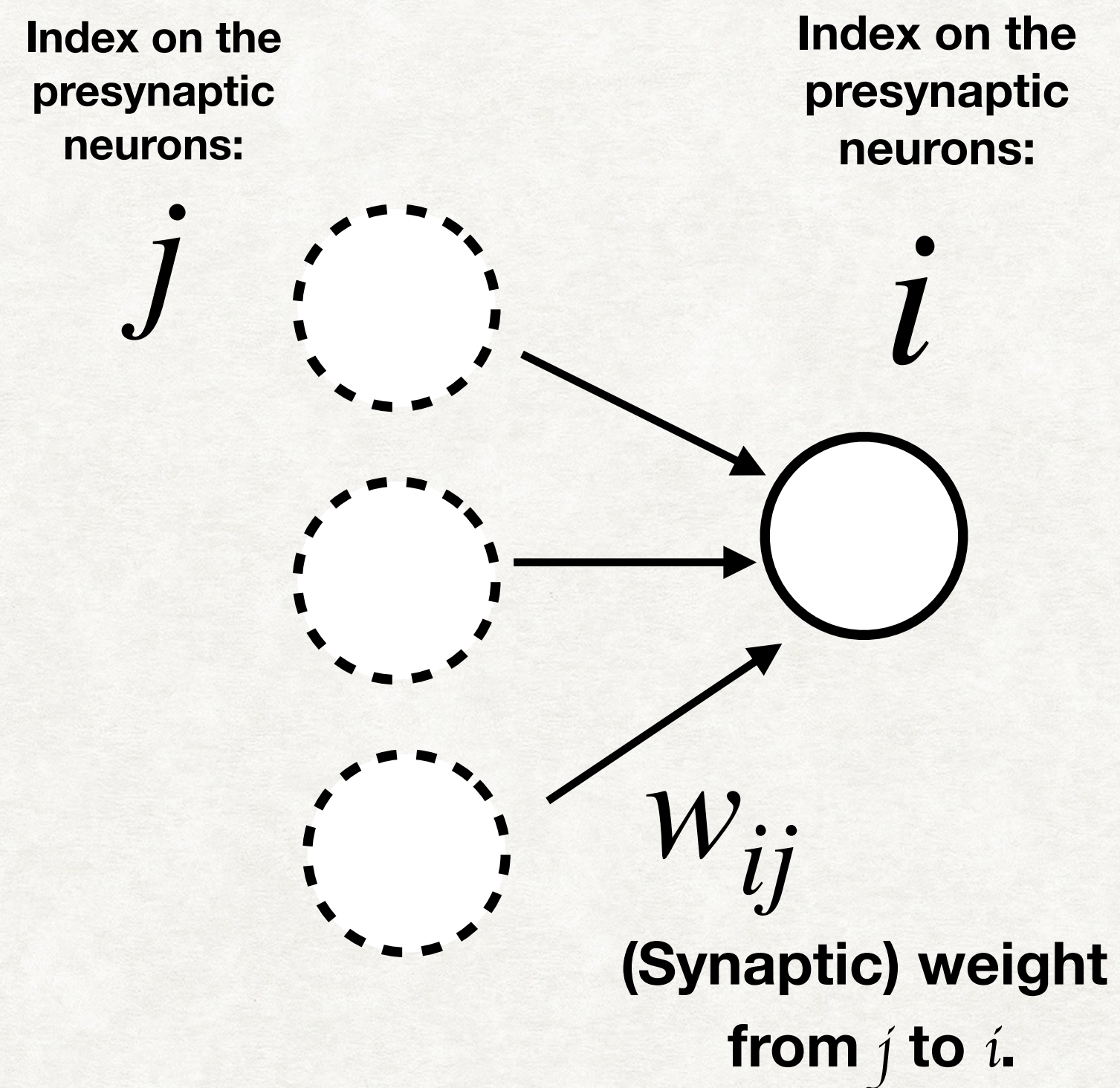
Find parameters (weights) that minimise the Loss function.

$$\Delta \mathbf{W}_{lk} \propto - \frac{\partial E}{\partial \mathbf{W}_{lk}}$$

$$\Delta \mathbf{W} \propto - \nabla_{\mathbf{W}} E$$

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

SINGLE NEURON



$$y = ax + b$$

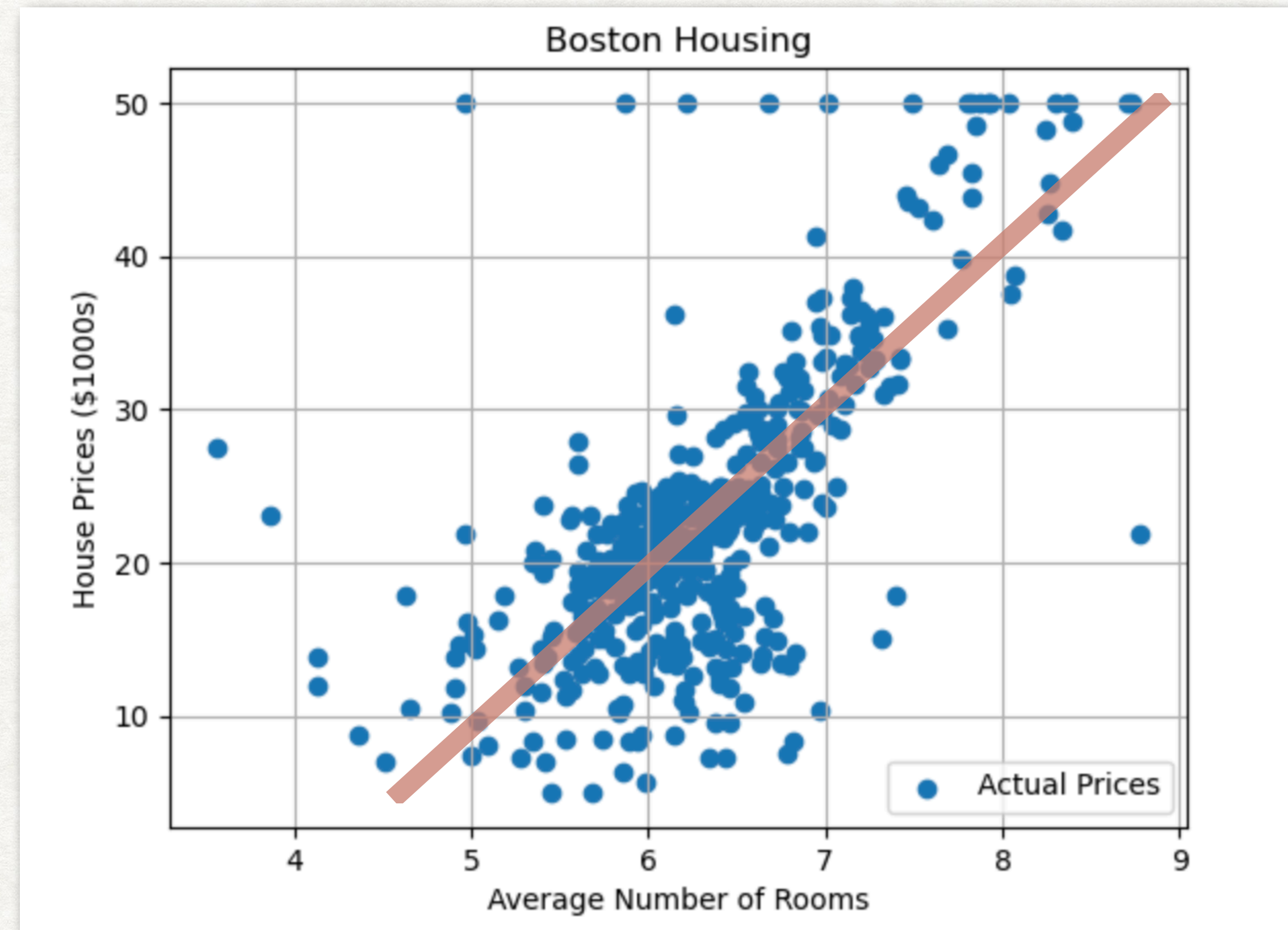
$$h_i = \sum_j w_{ij} x_j + b_i$$

$$x_i = f(h_i)$$

Activation function f , sigmoidal or ReLU

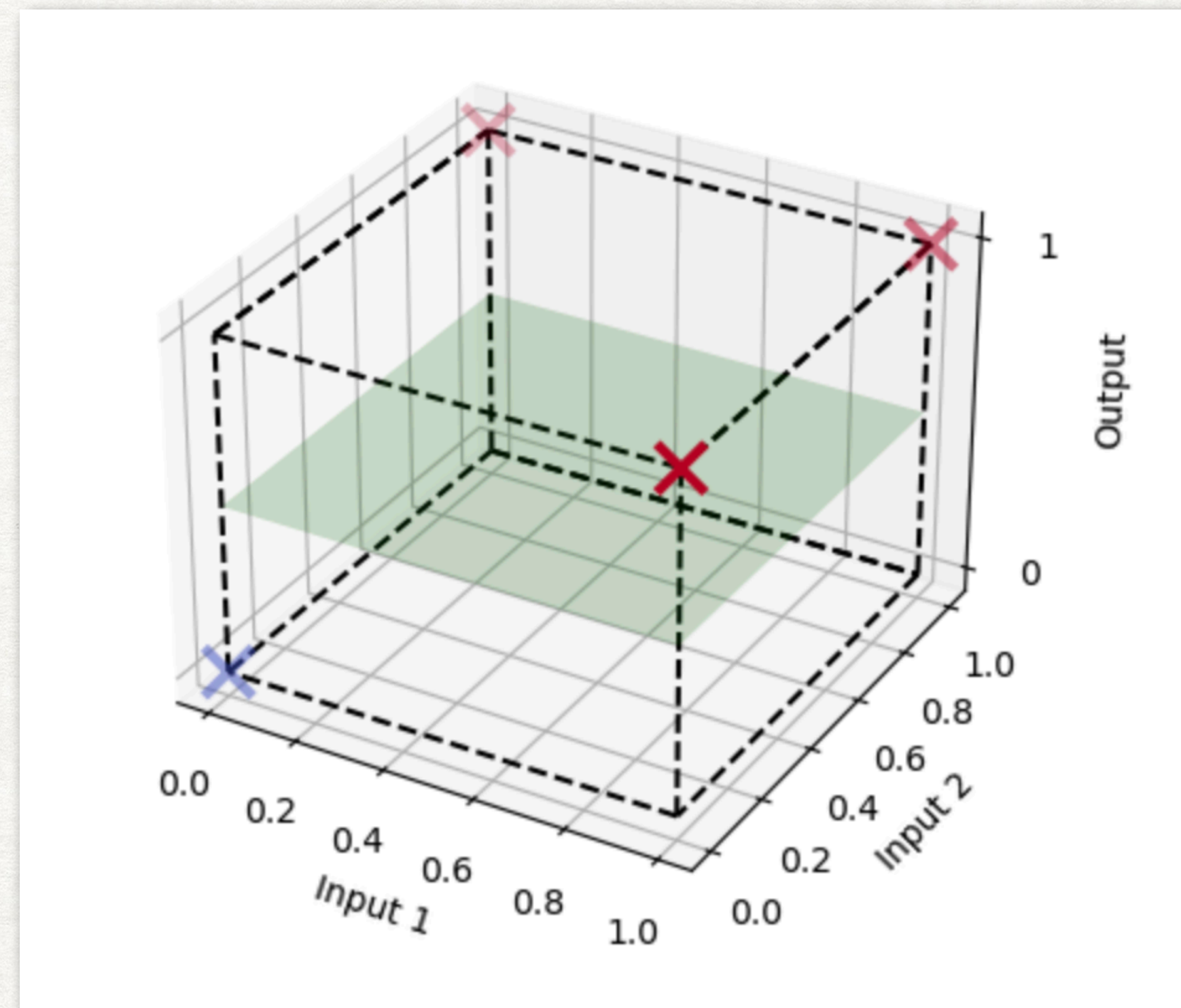
ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

LINEAR MODEL



ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

LINEAR SEPARABILITY



ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

GRADIENT DESCENT

$$E = \frac{1}{2n} \sum_i \sum_{\mathbf{x}} \left(y_i^t(\mathbf{x}) - y_i(\mathbf{x}; \mathbf{W}) \right)^2$$

$$\Delta w_{ij} = -\eta \frac{\partial E}{\partial w_{ij}}$$

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

GRADIENT DESCENT

$$E = \frac{1}{2n} \sum_i \sum_{\mathbf{x}} \left(y_i^t(\mathbf{x}) - y_i(\mathbf{x}; \mathbf{W}) \right)^2$$

$$\frac{\partial E}{\partial w_{ij}} = -\frac{1}{n} \sum_{\mathbf{x}} \left(y_i^t(\mathbf{x}) - y_i(\mathbf{x}) \right) \frac{\partial y_i(\mathbf{x})}{\partial w_{ij}}$$

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

GRADIENT DESCENT

$$E = \frac{1}{2n} \sum_i \sum_{\mathbf{x}} \left(y_i^t(\mathbf{x}) - y_i(\mathbf{x}; \mathbf{W}) \right)^2$$

$$\frac{\partial E}{\partial w_{ij}} = -\frac{1}{n} \sum_{\mathbf{x}} \left(y_i^t(\mathbf{x}) - y_i(\mathbf{x}) \right) \frac{\partial y_i(\mathbf{x})}{\partial w_{ij}}$$

$$y_i(\mathbf{x}) = f \left(\sum_j w_{ij} x_j + b_i \right)$$

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

GRADIENT DESCENT

$$E = \frac{1}{2n} \sum_i \sum_{\mathbf{x}} \left(y_i^t(\mathbf{x}) - y_i(\mathbf{x}; \mathbf{W}) \right)^2$$

$$\frac{\partial E}{\partial w_{ij}} = -\frac{1}{n} \sum_{\mathbf{x}} \left(y_i^t(\mathbf{x}) - y_i(\mathbf{x}) \right) \frac{\partial y_i(\mathbf{x})}{\partial w_{ij}}$$

$$\Delta w_{ij} = -\eta \frac{1}{n} \sum_{\mathbf{x}} \left(y_i(\mathbf{x}) - y_i^t(\mathbf{x}) \right) y_i(\mathbf{x})(1 - y_i(\mathbf{x}))x_j$$

f: sigmoid

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

GRADIENT DESCENT

$$E = \frac{1}{2n} \sum_i \sum_{\mathbf{x}} \left(y_i^t(\mathbf{x}) - y_i(\mathbf{x}; \mathbf{W}) \right)^2$$

$$\frac{\partial E}{\partial w_{ij}} = -\frac{1}{n} \sum_{\mathbf{x}} \left(y_i^t(\mathbf{x}) - y_i(\mathbf{x}) \right) \frac{\partial y_i(\mathbf{x})}{\partial w_{ij}}$$

$$\Delta b_i = -\eta \frac{1}{n} \sum_{\mathbf{x}} \left(y_i(\mathbf{x}) - y_i^t(\mathbf{x}) \right) y_i(\mathbf{x})(1 - y_i(\mathbf{x}))$$

f: sigmoid

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

GRADIENT DESCENT

$$E = \frac{1}{2n} \sum_i \sum_{\mathbf{x}} \left(y_i^t(\mathbf{x}) - y_i(\mathbf{x}; \mathbf{W}) \right)^2$$

$$\frac{\partial E}{\partial w_{ij}} = -\frac{1}{n} \sum_{\mathbf{x}} \left(y_i^t(\mathbf{x}) - y_i(\mathbf{x}) \right) \frac{\partial y_i(\mathbf{x})}{\partial w_{ij}}$$

$$\Delta w_{ij} = \eta \frac{1}{n} \sum_{\mathbf{x}} \left(y_i^t(\mathbf{x}) - y_i(\mathbf{x}) \right) \cdot \mathbf{ReLU}' \left(\sum_j w_{ij} x_j + b_i \right) \cdot x_j$$

f: ReLU

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

GRADIENT DESCENT

$$E = \frac{1}{2n} \sum_i \sum_{\mathbf{x}} \left(y_i^t(\mathbf{x}) - y_i(\mathbf{x}; \mathbf{W}) \right)^2$$

$$\frac{\partial E}{\partial w_{ij}} = -\frac{1}{n} \sum_{\mathbf{x}} \left(y_i^t(\mathbf{x}) - y_i(\mathbf{x}) \right) \frac{\partial y_i(\mathbf{x})}{\partial w_{ij}}$$

$$\Delta b_i = \eta \frac{1}{n} \sum_{\mathbf{x}} \left(y_i^t(\mathbf{x}) - y_i(\mathbf{x}) \right) \cdot \mathbf{ReLU}' \left(\sum_j w_{ij} x_j + b_i \right)$$

f: ReLU

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

EFFICIENT IMPLEMENTATION

$$y_i(\mathbf{x}) = f \left(\sum_j w_{ij} x_j + b_i \right)$$

X = ... # Input data matrix, shape (batch_size, input_dim)
y_t = ... # Target values matrix, shape (batch_size, output_dim)
y = ... # Computed outputs matrix, shape (batch_size, output_dim)
W = ... # Weight matrix, shape (input_dim, output_dim)
b = ... # Bias vector, shape (output_dim,)
learning_rate = ... # Learning rate, a scalar

Always check DIMENSIONS

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

EFFICIENT IMPLEMENTATION

$$y_i(\mathbf{x}) = f \left(\sum_j w_{ij} x_j + b_i \right)$$

Code Snippet:

Compute the linear transformation using matrix multiplication

linear_output = np.matmul(X, W) + b

Apply activation function element-wise

y_i = activation_function(linear_output)

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

EFFICIENT IMPLEMENTATION

$$\Delta w_{ij} = -\eta \frac{1}{n} \frac{\partial E}{\partial w_{ij}} = -\eta \frac{1}{n} \sum_{\mathbf{x}} (y_i(\mathbf{x}) - y_i^t(\mathbf{x})) y_i(\mathbf{x})(1 - y_i(\mathbf{x})) x_j,$$

Code Snippet:

Compute updates for weights

```
update_weights = -learning_rate * np.matmul(X.T, error_times_derivative) /  
X.shape[0]
```

f: sigmoid

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

EFFICIENT IMPLEMENTATION

$$\Delta b_i = -\eta \frac{1}{n} \sum_{\mathbf{x}} (y_i(\mathbf{x}) - y_i^t(\mathbf{x})) y_i(\mathbf{x})(1 - y_i(\mathbf{x}))$$

Code Snippet:

Compute updates for biases

Sum over the batch dimension (axis=0) to get individual updates for each output neuron - in case there are more than one

**update_biases = -learning_rate * np.sum(error_times_derivative, axis=0) /
X.shape[0]**

f: sigmoid

ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

TRAINING, VALIDATION AND TESTING



parameters



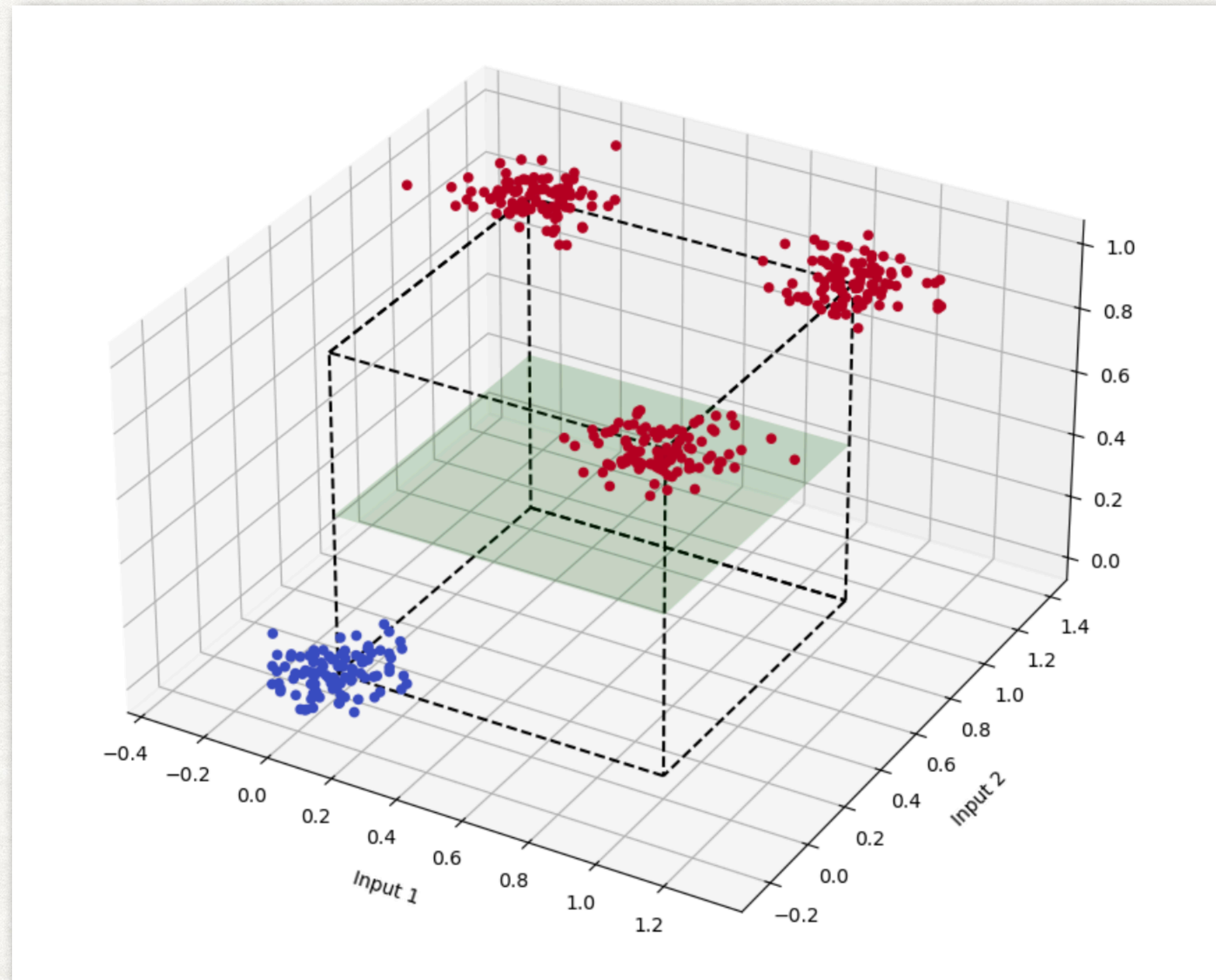
hyper-parameters



generalisation

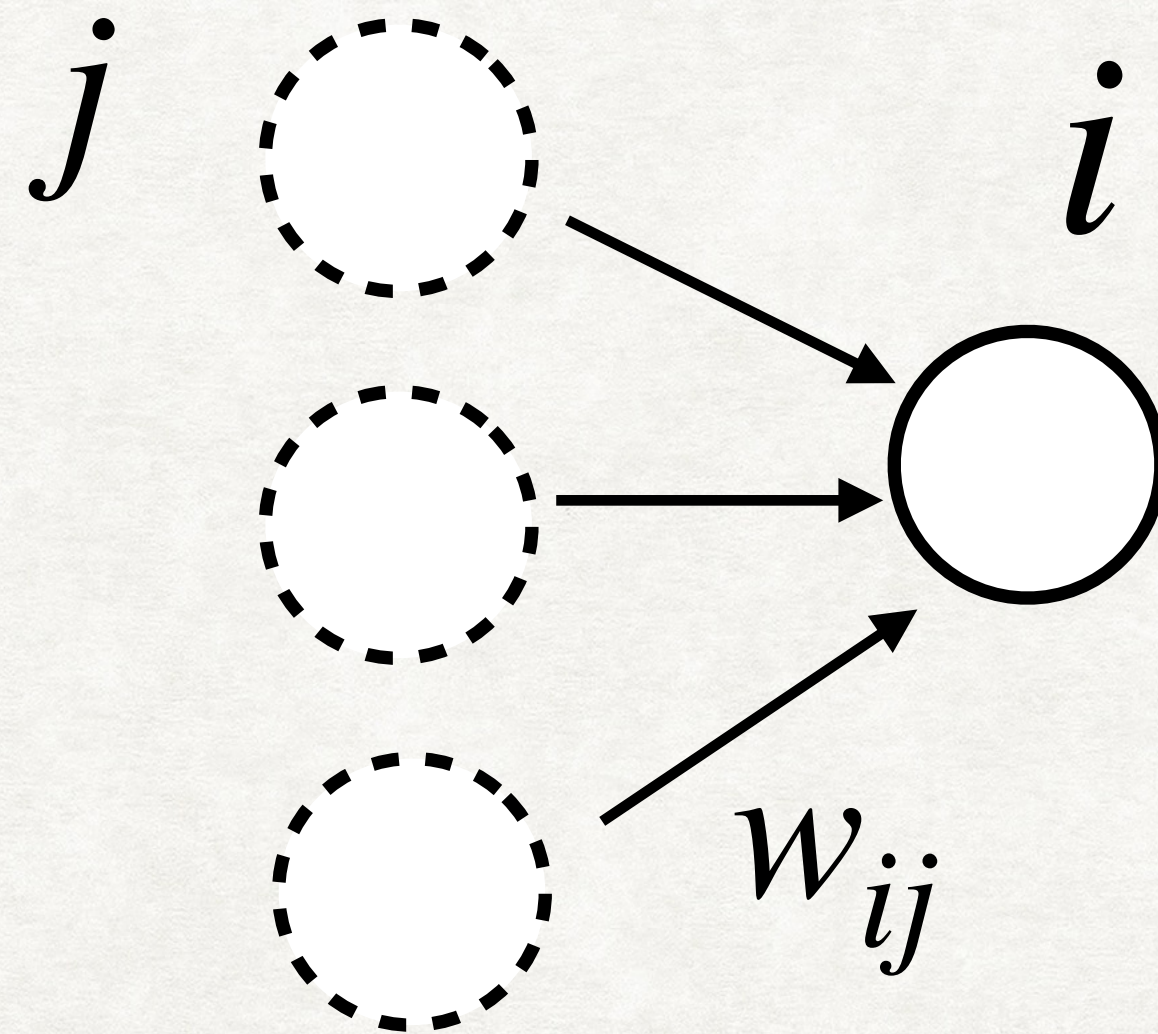
ARTIFICIAL NEURAL NETWORKS AS FUNCTION APPROXIMATIONS

EXAMPLE



NEXT SESSION

REPLACE Q-VALUE TABLE WITH ANN



[0.000,	0.002,	]
[0.000,	0.012,	]
[0.000,	0.069,	]
[0.000,	0.261,	]
[0.000,	0.651,	]
[0.000,	0.000,	]

THANK YOU!